HR AI Assistant V2 Documentation

HR AI Assistant V2
==========

1. Project Overview

This project is an HR policy question-answering assistant built as a web application. It reads HR policy PDFs from the `data/` directory, converts the PDF text into search-ready chunks, and uses OpenAI to generate answers based on the extracted policy content.

The application is designed to support two roles:
- Administrator: can upload HR policy PDFs and manage the knowledge base.
- Employee: can ask questions and receive policy-aware answers.

The solution is built as a FastAPI backend server with a static HTML/JavaScript frontend. Answers are cached in SQLite to reduce repeated OpenAI usage.

2. End-to-End Pipeline

The pipeline follows these steps:

1. User visits the web app in a browser.
2. The browser loads `static/index.html` and CSS from `assets/style.css`.
3. The user logs in as an administrator or employee.
4. When the user submits a question, the browser sends a request to `/api/ask`.
5. The backend checks whether that question has an existing cached answer in SQLite (`database/hr_ai.db`).
6. If the answer is cached, return it immediately.
7. If not cached:
- The backend extracts text from the HR PDFs using `utils/pdf_reader.py`.
- The text is split into smaller chunks.
- The backend selects the most relevant chunks for the question.
- The selected chunks and question are sent to OpenAI by `utils/openai_helper.py`.
- OpenAI returns an answer based on the HR policy context.
- The answer is stored in the cache and returned to the browser.

8. The frontend displays the answer in the chat history.

When an administrator uploads a PDF via the UI, the backend saves the file to `data/`, refreshes the cached PDF text, and makes the new content available to future questions.

3. Conceptual Framework

Core concepts used in this project:

- HR Knowledge Base: The PDF documents are the source of truth for HR policies.
- Document ingestion: The application reads PDF text and prepares it for querying.

- Chunking: Large documents are split into smaller text blocks so the model can focus on relevant fragments.
- Relevance scoring: The application scores chunks by keyword overlap to select the best context.
- Model prompt engineering: The OpenAI request instructs the model to answer only from the provided policy text.
- Caching: Answers are stored in SQLite to avoid repeated API calls for the same question.
- Backend server: FastAPI acts as a secure control plane for the AI model.
- Frontend UI: Static HTML/JavaScript provides an accessible chat and upload interface.

This architecture is beginner-friendly because it separates concerns clearly:
- frontend handles user interaction,
- backend handles data and AI calls,
- utilities handle PDF processing,
- database handles caching.

4. Major Files Explained

----

app.py

This is the main backend server. Its responsibilities are:

- Load environment variables from `.env`.
- Validate that `OPENAI_API_KEY` is available.
- Create the `data/` folder if needed.
- Mount static assets so the browser can load the web app.
- Initialize the SQLite cache during startup.
- Serve the HTML page at `/`.
- Provide `/api/docs` to list HR PDFs.
- Provide `/api/ask` to answer questions.
- Provide `/api/upload` for admin-only document uploads.

Key sections in `app.py`:

- `AskRequest` model: defines the request schema expected by `/api/ask`.
- `startup_event()`: ensures the database exists and preloads the PDF text.
- `list_documents()`: returns PDF file names for the UI document list.
- `ask_question(request)`: the main question-answering flow.
- `upload_document(file)`: saves uploaded PDF files and refreshes cached text.
- `get_relevant_chunks()`: finds the top matching text chunks based on keyword overlap.

Details of `ask_question(request)`:
- Strips whitespace from the question.
- Rejects empty questions.
- Retrieves a cached answer from SQLite if available.
- Otherwise, reads the PDF text chunks.
- Selects the best matching chunks.
- Calls `ask_openai()` with the selected context.
- Saves the answer in the cache.

- Returns a JSON response with `answer` and `cached` status.

utils/openai_helper.py

This module handles communication with OpenAI.

Key points:
- Uses `OpenAI(api_key=api_key)` to create a client.
- Builds a system prompt that instructs the model to answer only from the provided policy content.
- Builds a user prompt that includes the selected HR policy text and the question.
- Sends a chat completion request with `model='gpt-4o-mini'`.
- Uses temperature `0.2` for more deterministic answers.
- Returns the model answer text.
- Handles API errors such as rate limits and service unavailability.

Why this is important:
- The helper isolates AI-specific logic from the rest of the application.
- The prompt contains explicit instructions forcing the model to rely on policy text.
- Error handling ensures the user receives a helpful message instead of an unhandled crash.

utils/pdf_reader.py

This module reads HR policy PDFs and converts them into text chunks.

Key behavior:
- `split_text(text, chunk_size=800)`: splits long text into 800-character chunks.
- `read_all_pdfs(folder_path)`: reads every `.pdf` file in the folder.
- Uses `pdfplumber` to open each PDF and extract text from every page.
- Concatenates the text from all pages and splits it into chunks.
- Uses `functools.lru_cache` to avoid re-reading PDFs on every request.

Why chunking is used:
- Large documents may exceed model input limits.
- Smaller chunks make it easier to select the most relevant portion.
- It enables context selection based on keyword similarity.

database/database.py

This module manages a local SQLite cache.

Key functions:
- `create_database()`: creates the database file and the `answer_cache` table.
- `get_cached_answer(question)`: returns a previously-stored answer for the exact question.
- `save_answer(question, answer)`: inserts or updates the cache.

Why caching is useful:
- It reduces OpenAI usage for repeated questions.
- It improves user response time.

- It stores answers permanently in `database/hr_ai.db`.

static/index.html

This file is the frontend user interface.

Main sections:
- Sidebar with branding, status, and document list.
- Login screen for `admin` and `employee`.
- Chat interface for asking questions.
- Admin upload panel to upload HR policy PDFs.

How it works:
- The user logs in with one of two hard-coded accounts.
- The frontend shows or hides the admin upload controls depending on user role.
- When the user asks a question, the browser sends a POST request to `/api/ask`.
- When an admin uploads a PDF, the browser sends the file to `/api/upload`.
- The UI displays responses from the backend and updates document status.

assets/style.css

This stylesheet controls the application's appearance.

Important visual rules:
- Grid layout for sidebar and main panel.
- Responsive design for smaller screens.
- Styled chat message bubbles.
- Modern card and button appearance.

Additional project files

- `.env.example`: shows how to configure `OPENAI_API_KEY`.
- `requirements.txt`: lists required Python packages.
- `README.md`: provides setup and run instructions.

5. How the system works as a whole

The full system is organized into layers:

- Data Layer: `data/` contains the HR policy PDFs.
- Ingestion Layer: `utils/pdf_reader.py` extracts text from PDFs.
- Search Layer: `app.py` selects relevant text chunks from those PDFs.
- Model Layer: `utils/openai_helper.py` sends the selected context and question to OpenAI.
- Cache Layer: `database/database.py` stores question/answer pairs.
- UI Layer: `static/index.html` and `assets/style.css` let users interact with the assistant.

The backend acts as the control plane for the OpenAI model, which is the most secure and maintainable

design for this architecture.

6. Beginner-Friendly explanation

If you are new to this type of project, think of it as a digital HR assistant with these parts:

- A website where employees can type questions.
- A hidden server that reads the questions and finds the right HR policy text.
- A language model that turns those policy fragments into a helpful answer.
- A database that remembers answers so the system does not call the AI again for the same exact question.

The server keeps the API key hidden, which protects your OpenAI credentials from being exposed in the browser.

7. Running the project

---

1. Install dependencies:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
```

2. Create `.env` from `.env.example` and add your key:

```
OPENAI_API_KEY=your_openai_api_key_here
```

3. Start the app:

```
python app.py
```

4. Open `http://localhost:8000` in your browser.

8. Notes and future improvements

Possible improvements:

- Replace hard-coded login with real authentication.
- Add better relevance scoring using embeddings.
- Add source citation and document references.
- Secure the upload endpoint with admin authentication.
- Add pagination for large chat histories.
- Improve prompt engineering for better factual accuracy.

This documentation is designed so a beginner can read the file and understand how the app is structured, how data flows through it, and why each major file exists.